MEV in Decentralised Exchanges

Continuous vs. Batch Clearing

Sanoj S Vijendra
Roll No: 22B0916

April 16, 2026

Outline

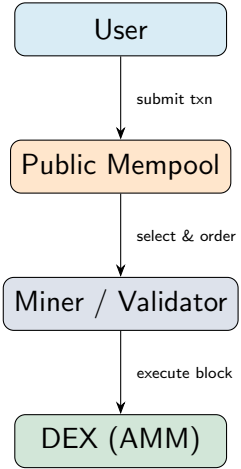
1. Decentralised Exchanges
2. Constant-Product AMM
3. Maximal Extractable Value (MEV)
4. Payoff Analysis
5. Clearing Mechanisms
6. Setup and Results
7. References

Decentralised Exchanges (DEXs)

Token swapping via **onbchain smart contracts** no central intermediary

Transactions are broadcast to a public **peer-to-peer mempool** before inclusion in a block

The public mempool creates **informational asymmetries** that rational actors can exploit



Key difference from CEXs

In a centralised exchange, the order book is private. On a DEX, *every pending trade is visible to everyone*.

Constant-Product Automated Market Maker

Reserve invariant

$$r_A \cdot r_B = k \quad \text{(constant)}$$

Output of a swap of Δ_A units of token A:

$$\boxed{\Delta_B = \frac{r_B \cdot \Delta_A}{r_A + \Delta_A}}$$

Spot price of A in terms of B:

$$P(r_A, r_B) = \frac{r_B}{r_A}$$

After the trade: new reserves are $(r_A + \Delta_A,\ r_B - \Delta_B)$.

Price Impact

Larger $\Delta_A \Rightarrow$ the reserves move further along the curve $\Rightarrow$ worse effective price for the trader. This is called **slippage**.

Slippage definition

$$\text{slip} = 1 - \frac{\hat{\Delta}_B}{\Delta_B^{\text{fair}}} \in [0, 1]$$

where Δ_B^{fair} is the output the user expected at time of submission.

Miner's Extractable Value (MEV)

Definition [1]

MEV is the **surplus value** that miners/validators can capture by selectively *reordering*, *inserting*, or *censoring* transactions within a block.

Forms of MEV on DEXs

Frontrunning — insert own trade *before* a detected profitable user trade

Sandwich attack — bracket a user trade with a buy *before* and sell *after*

Backrunning — exploit the price change *after* a large trade

Why it matters

Directly **reduces user welfare** (worse execution prices)

Wastes gas in **Priority Gas Auctions** (PGAs)

Creates **consensus-layer instability**: high MEV incentivises chain reorgs [1]

Daian et al. [1] documented over $6M USD in pure MEV revenue on Ethereum in a 20-month

The Sandwich Attack

Mechanism — given a user trade x_i of size δ_i (buying B with A):

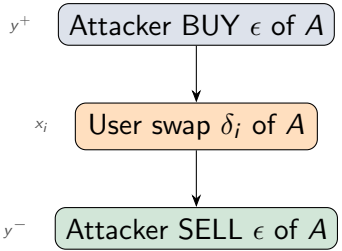
(1) Attacker buys ϵ of A *before* x_i
$\Rightarrow$ raises price; depletes r_B

(2) User trade x_i executes at the *worse* price
$\Rightarrow$ user receives $\hat{\Delta}_B < \Delta_B^{\text{fair}}$

(3) Attacker sells ϵ of A *after* x_i
$\Rightarrow$ price partially recovers; attacker profits

Attacker profit:

$$\pi^{\text{sandwich}} = \Delta_B^{y^+} - \Delta_B^{y^-} > 0$$



User loss

$$\text{slip}_i = 1 - \frac{\hat{\Delta}_B}{\Delta_B^{\text{fair}}} > 0$$

Priority Gas Auctions (PGAs)

MEV opportunities are **broadcast publicly** via the gossip network

Competing bots race to be first: submit the same transaction repeatedly with **increasing gas prices**

The miner includes the **highest-gas** transaction $\Rightarrow$ bots bid up

Key empirical findings [1]

PGAs have **all-pay** characteristics: losing bids still consume gas

Mean inter-bid time fell $0.8s \rightarrow 0.1s$ (2018–2019)

Gas waste

Gas consumed by *losing* bids in a PGA is pure deadweight loss — value destroyed, not transferred.

Who participates in PGAs?

Low-latency bots: well-connected, fast reaction

High-latency bots: slower, often outbid

Ordinary users: unaware; pay gas but lose priority

User Payoff

User payoff for a swap

$$\pi_i^U = v_i \cdot \underbrace{\frac{\hat{\Delta}_B^i}{\Delta_B^{\text{fair}}}}_{\text{execution quality}} - g_i \cdot c_{\text{gas}} - \mathbb{1}[\text{reverted}] \cdot g_i^{\text{used}} \cdot c_{\text{gas}}$$

Terms explained

v_i — user's private valuation for executing the trade

$\hat{\Delta}_B^i / \Delta_B^{\text{fair}}$ — ratio of actual to expected output; equals 1 with no MEV

$g_i \cdot c_{\text{gas}}$ — gas fee paid regardless of outcome

Three user types in simulation

Ordinary users — submit passively; vulnerable to MEV

Low-latency bots — fast propagation; can react to mempool events

High-latency bots — slower; often outbid in PGAs

Miner Payoff

Total miner payoff for a block

$$\pi_M = \underbrace{\sum_{i=1}^{k} g_{\sigma(i)} \cdot \mathrm{gas}_{\sigma(i)}}_{\text{gas fees from user txns}} + \underbrace{\sum_{y_j \in \mathcal{I}} \pi_M^{(j)}}_{\text{MEV from inserted txns}} + B_{\mathrm{block}}$$

Miner archetypes

Honest miner: orders by gas price.

Attacker: inserts y^+, y^- around large user swaps

Sandwich MEV (explicit formula)

$$\pi_M^{\mathrm{MEV}} = \frac{r_B\,\epsilon}{r_A + \epsilon} - \frac{(r_B - \Delta_B^{y^+} - \hat{\Delta}_B^{i})\,\epsilon}{r_A + \delta_i}$$

Two Clearing Mechanisms

Continuous AMM Clearing

Transactions execute **sequentially** in miner-chosen order

Each trade updates reserves *immediately*

Miner can insert trades **between** user transactions

Sandwich attacks are **profitable**

Gas price $\Rightarrow$ determines priority

Batch Auction Clearing [2]

All txns in a block execute **simultaneously** at a **uniform clearing price**

No single transaction can be inserted *between* others

Sandwich attacks **infeasible within a batch**

Gas price no longer determines within batch priority

Budish, Cramton and Shim [2] showed that batch clearing eliminates the value of sub-interval speed advantages by treating all orders within a batch symmetrically.

Theoretical Welfare Predictions

cc

Metric	Continuous AMM	Batch Clearing
User slippage (avg.)	High	Low (uniform price)
Miner MEV revenue	High	Low
Gas fee waste (PGAs)	High	Low
Trade execution latency	High	Low
Frontrunning exposure	High	Low (within batch)
Revert rate	Higher	Lower

Batch clearing is not MEV-*free*: between-batch manipulation remains possible, but its magnitude is bounded by price movement across consecutive blocks much smaller than within-transaction sandwich slippage.

Implementation

Events: GENERATE TRANSACTION, START MINING, FINISH MINING, RECEIVE BLOCK, CLEAR MEMPOOL, , LOOKUP MEMPOOL.

Setup:

Single Dex: Two tokens A and B, 1000000 and 1000000000 respectively in liquidity pool.

Mean block time: 12 sec.

Miner: Honest, Attacker (in ratio 7:3).

Users: (Low latency bots : High latency bots : Users :: 1:1:8), total 100 in number.

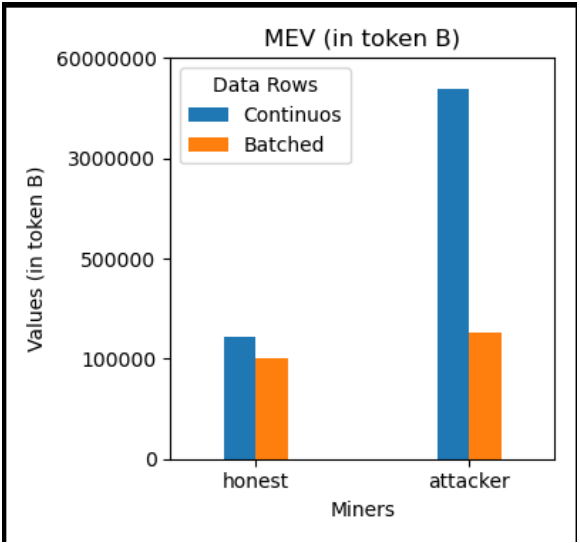
Mean txn generation time: 1 sec.

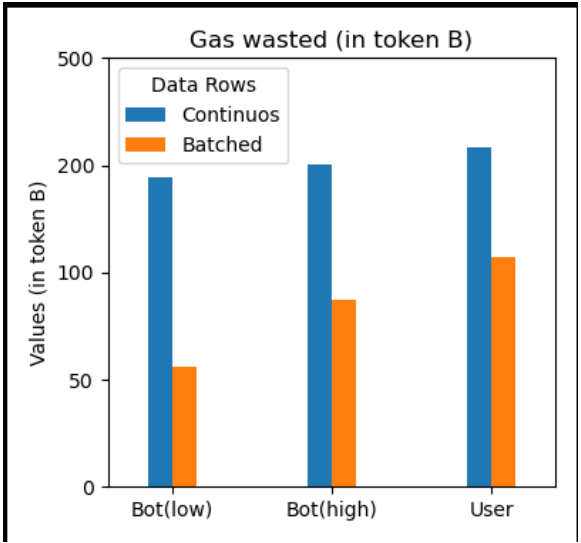
Total simulation time: 3600 sec (around 300 blocks).

Tolerance: 5% for users, 1% for bots. Minimum bid increase of 12.5%.

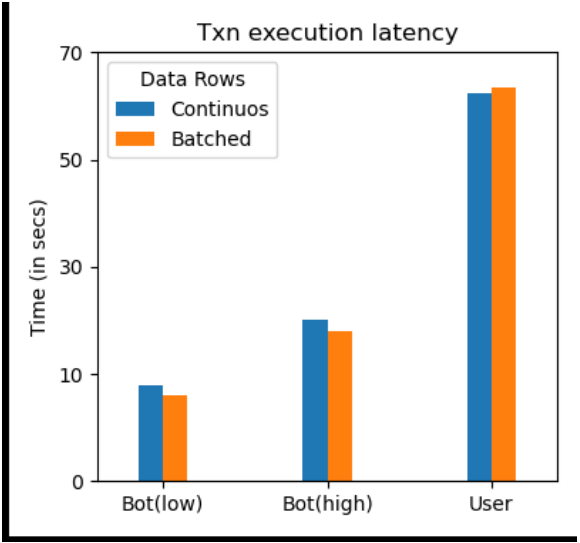
Relaxation: Instead of timebased batch, implemented block based. All txns executed in block at same rate.

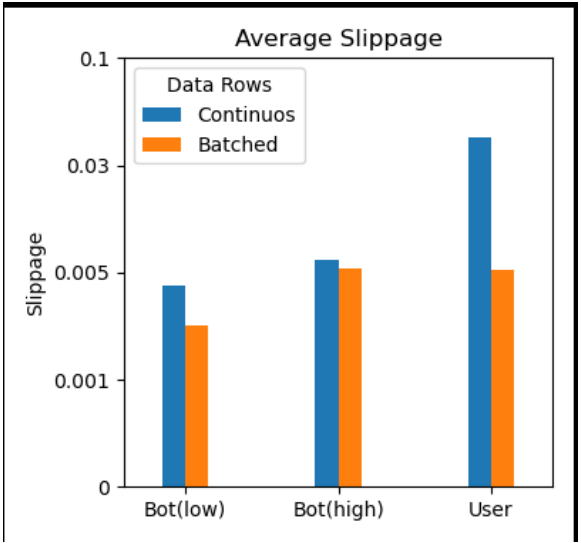
Results (1/3)


MEV (in token B)
Data Rows
Continuos
Batched
Values (in token B)
60000000
3000000
500000
100000
0
honest
attacker
Miners

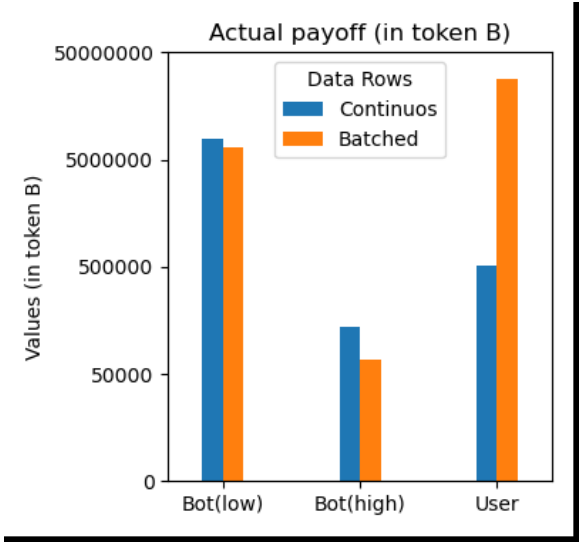

Gas wasted (in token B)
Data Rows
Continuos
Batched
Values (in token B)
500
200
100
50
0
Bot(low)
Bot(high)
User

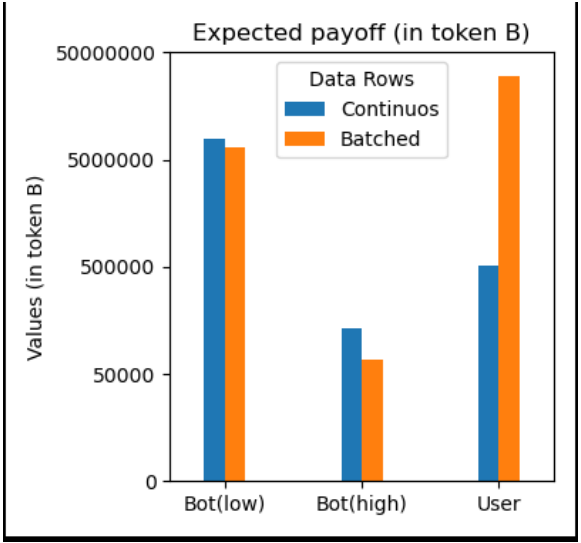
Results (2/3)


Txn execution latency
Data Rows
Continuos
Batched
Time (in secs)
70
50
30
10
0
Bot(low)
Bot(high)
User


Average Slippage
Data Rows
Continuos
Batched
Slippage
0.1
0.03
0.005
0.001
0
Bot(low)
Bot(high)
User

Results (3/3)


Actual payoff (in token B)
Values (in token B)
50000000
5000000
500000
50000
0
Data Rows
Continuos
Batched
Bot(low)
Bot(high)
User


Expected payoff (in token B)
Values (in token B)
50000000
5000000
500000
50000
0
Data Rows
Continuos
Batched
Bot(low)
Bot(high)
User

References I

P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels.

Flash Boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralised exchanges.

arXiv:1904.05234, 2019.

E. Budish, P. Cramton, and J. Shim.

The high-frequency trading arms race: Frequent batch auctions as a market design response.

The Quarterly Journal of Economics, 130(4):1547–1621, 2015.

References II

Z. Alipanahloo, K. Zhang, and E. Awosika.

Quantifying fairness granularity as a fair ordering policy towards MEV mitigation for rollups.

In *IEEE International Conference on Blockchain*, 2024.